GitHub for Beginners: A Practical Guide to Collaboration and Open Source

Your First Steps into Collaborative Open Source

Presenter： Karthik Lu

Contents


01
Introduction
to GitHub

What is GitHub and Why It Matters

Introduction to GitHub

GitHub is a web-based platform that hosts code repositories and enables developers to collaborate on projects. It uses Git version control to track changes, manage versions, and coordinate work among team members. GitHub is essential because it allows you to store your code safely in the cloud, collaborate with others seamlessly, track project history, and showcase your work to potential employers or clients. Whether you're working solo or in a team, GitHub provides the infrastructure for modern software development workflows.

Real-World Use Cases for GitHub

Software Development Teams

Teams use GitHub to collaborate on codebases, review each other's code, and merge updates safely. Multiple developers can work on different features simultaneously without conflicts.

Open Source Projects

Thousands of open source projects (like Linux, Python libraries, frameworks) are hosted on GitHub. Contributors worldwide can propose improvements, fix bugs, and add features.

Portfolio Building

Students and professionals showcase their projects on GitHub to demonstrate skills to recruiters and employers. A strong GitHub profile is valuable in the tech industry.

Learning and Documentation

Beginners learn programming by studying open source code. Documentation and tutorials are often stored in GitHub repositories.

Creating Your GitHub Account

Sign Up Process

Visit github.com and click "Sign up." Enter your email, create a password, and choose a username (this will be part of your profile URL, so choose wisely). Verify your email and complete the onboarding.

Dashboard Overview

Once logged in, you'll see the Dashboard with your repositories, activity feed, and recommendations.

Navigation Bar

The top navigation bar has a search function, notifications, and profile menu.

Main Menu Items

The main menu includes: Repositories (your projects), Stars (bookmarked repos), Issues (tasks and bugs), Pull Requests (code reviews), and Settings (account preferences).

Getting Started

Familiarize yourself with these sections before creating your first repository.


02
Working with
Repositories

Understanding Repositories

What is a Repository

A repository (or "repo") is a folder that contains your project files, including code, documentation, and version history. Think of it as a project container where all changes are tracked. Each file's history is preserved, allowing you to see who made what changes and when.

Repository Storage

Repositories can be stored locally on your computer or hosted remotely on GitHub (or both).

Typical Contents

A typical repository includes a README file (project description), source code files, configuration files, and sometimes documentation folders.

Creating Your First Repository

01

Step 1: Navigate to New Repository

Click the "+" icon in the top-right corner and select "New repository."

02

Step 2: Name Your Repository

Choose a descriptive name (e.g., "my-first-project" or "portfolio-website"). GitHub suggests lowercase with hyphens instead of spaces.

03

Step 5: Initialize Repository

Check "Add a README file" to include a template. Optionally add a .gitignore file and license.

04

Step 3: Add a Description

Write a brief description of what your project does (optional but recommended).

05

Step 6: Create Repository

Click "Create repository." Your repo is now live on GitHub.

06

Step 4: Choose Visibility

Select Public (visible to everyone) or Private (visible only to you and collaborators you invite).

Public vs. Private Repositories

Public Repositories

Visible to anyone on the internet. Anyone can see your code, fork it, and contribute. Ideal for open source projects, portfolio pieces, and sharing knowledge. No cost limitation—you can create unlimited public repos. Great for learning and community engagement.

Private Repositories

Only you and people you explicitly invite can access them. Useful for confidential projects, client work, and personal experiments. GitHub provides free private repositories for all users. Private repos are essential for protecting intellectual property and work-in-progress projects. You control who sees your code.

Adding Files and Understanding README.md

Add Files via Web Interface

To add files directly via GitHub's web interface: Open your repository, click "Add file," then "Create new file" or "Upload files." For creating files, type the filename (e.g., "index.html"), write or paste content, add a commit message (a brief description of your change), and click "Commit changes."

README Impact

A well-written README helps others understand your project instantly and increases the likelihood of collaboration and stars.

README.md Purpose

README.md is a special file that displays automatically on your repo's main page.

README.md Contents

It should include: project title, description, installation instructions, usage examples, and how to contribute.

Uploading Projects to GitHub

- **Web Upload**

  Click "Add file" → "Upload files," select multiple files from your computer, write a commit message, and commit. Best for small projects or quick updates.

- **Command Line (Git)**

  Clone your repository, add files locally using , commit using , and push using . This is the standard workflow for developers.

- **GitHub Desktop**

  Download GitHub Desktop app (a visual Git client), clone your repo, drag and drop files, write commit messages, and sync. Beginner-friendly alternative to command line.

- **Course Focus**

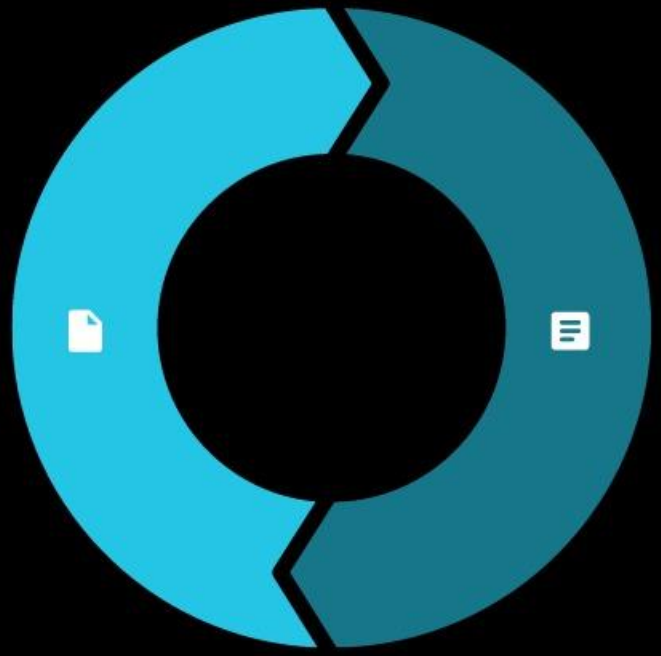
  For this course, focus on web upload and GitHub Desktop. As you advance, learn command-line Git for faster, more efficient workflows.

03 | Collaboration and Workflow

Understanding Forks and Branching

Fork

A fork is a personal copy of someone else's repository. When you fork a project, you get your own independent copy on GitHub. You can modify it without affecting the original. Forks are the foundation of open source contribution—you fork, make changes, then propose those changes back via a Pull Request.



Branching

A branch is a separate line of development within a repository. The default branch is usually "main" or "master." You create branches to work on features or fixes without affecting the main code. For example: create a "dark-mode" branch, develop the feature, test it, then merge it back to main.

Creating Branches via GitHub Web

01

Step 1: Open Your Repository

Navigate to your repo and click the branch dropdown (usually shows "main").

02

Step 2: Create New Branch

Type a new branch name in the search box (e.g., "add-navigation-menu") and select "Create branch."

03

Step 3: Switch to New Branch

The dropdown now shows your new branch. You're working on this branch, not the main code.

04

Step 4: Make Changes

Edit files, create new files, and commit changes to this branch.

05

Step 5: Prepare for Merge

Once your feature is complete, you'll create a Pull Request (described next) to merge this branch back to main. This protects the main branch from incomplete or broken code.

Pull Requests: Proposing Changes

01

What is a Pull Request?

A Pull Request (PR) is a request to merge your branch changes into another branch (usually main). It's a formal way to propose code changes and get feedback. When you create a PR, you explain what you changed and why.

02

PR Review Process

Team members review your code, leave comments, suggest improvements, and approve or request changes. PRs enable code quality control and knowledge sharing. Even on solo projects, PRs are good practice.

03

Creating a Pull Request

To create a PR: after committing changes to your branch, GitHub suggests creating a PR. Click "Compare & pull request," add a title and description, and click "Create pull request."

Reviewing, Approving, Merging PRs

Step 1: Review Code Changes

Open the PR and view the "Files changed" tab to see exact modifications (green = added, red = removed).

Step 2: Leave Comments

Click the "+" icon next to a line to comment or suggest changes. Use the "Conversation" tab for general discussion.

Step 3: Approve or Request Changes

Click "Review changes," select "Approve" (looks good), "Comment" (feedback without blocking), or "Request changes" (must fix before merging).

Step 4: Address Feedback

If changes are requested, make edits on the branch, commit, and push. The PR updates automatically.

Step 5: Merge

Once approved, click "Merge pull request." Choose merge strategy (usually "Create a merge commit") and confirm. Your branch is now merged into main.

Issues and Discussions for Collaboration

Issues

Use Issues to track bugs, feature requests, and tasks. Open the "Issues" tab, click "New issue," add a title and description, assign it to team members, add labels (e.g., "bug," "enhancement"), and set milestones if applicable. Issues provide a centralized place to discuss problems and solutions.

Discussions

A more relaxed space for questions, ideas, and community chat. Unlike Issues (structured tasks), Discussions are for broader conversations, feedback, and knowledge sharing. Both Issues and Discussions are essential for collaboration—they create transparency and keep communication documented within the project.

04 | Intermediate Features and Portfolio Building

Optimizing Your GitHub Profile

01

Profile Customization Steps

Click your profile icon → "Your profile" to customize it. Add a profile picture (professional headshot), write a bio (who you are and what you do), add links to your website or social media, and pin your best repositories (up to 6) on your profile for visitors to see immediately.

02

Special README Setup

Create a special repository with your GitHub username (e.g., "johndoe/johndoe") and add a README.md to it—this README displays on your profile and can include your skills, projects, and contact information.

03

Profile Maintenance

Keep your profile updated, maintain clean and well-documented repositories, and keep your commit history presentable. Recruiters often check GitHub profiles.

Introduction to GitHub Pages

Hosting Options

GitHub Pages allows you to host a free website directly from a GitHub repository. Each account gets one free site, and each repository can have a project site.

User Site Setup

Create a repository named "username.github.io" (replacing "username" with your GitHub username), add HTML/CSS/JavaScript files, and your site goes live at https://username.github.io.

Project Site Setup

Alternatively, create any repository and enable Pages in Settings to host a project site at https://username.github.io/repository-name.

Use Cases

This is perfect for hosting a portfolio website, project documentation, or personal blog—all free and version-controlled on GitHub.

Templates and Markdown Tips

Templates

GitHub provides templates for README files, issue templates, and pull request templates. When creating a new repository, you can choose templates that structure your project documentation. Templates ensure consistency and save time. Check the Templates section in your repo settings.

Markdown

Most GitHub files support Markdown—a simple formatting language. Use # for headers, bold for emphasis, , - for lists, for inline code, and for links. Markdown makes your documentation clean and readable. Practice Markdown in your README and other documentation files. Most importantly, write clear, concise, and well-organized documentation.

Exploring Open Source Learning

01 GitHub as Learning Resource

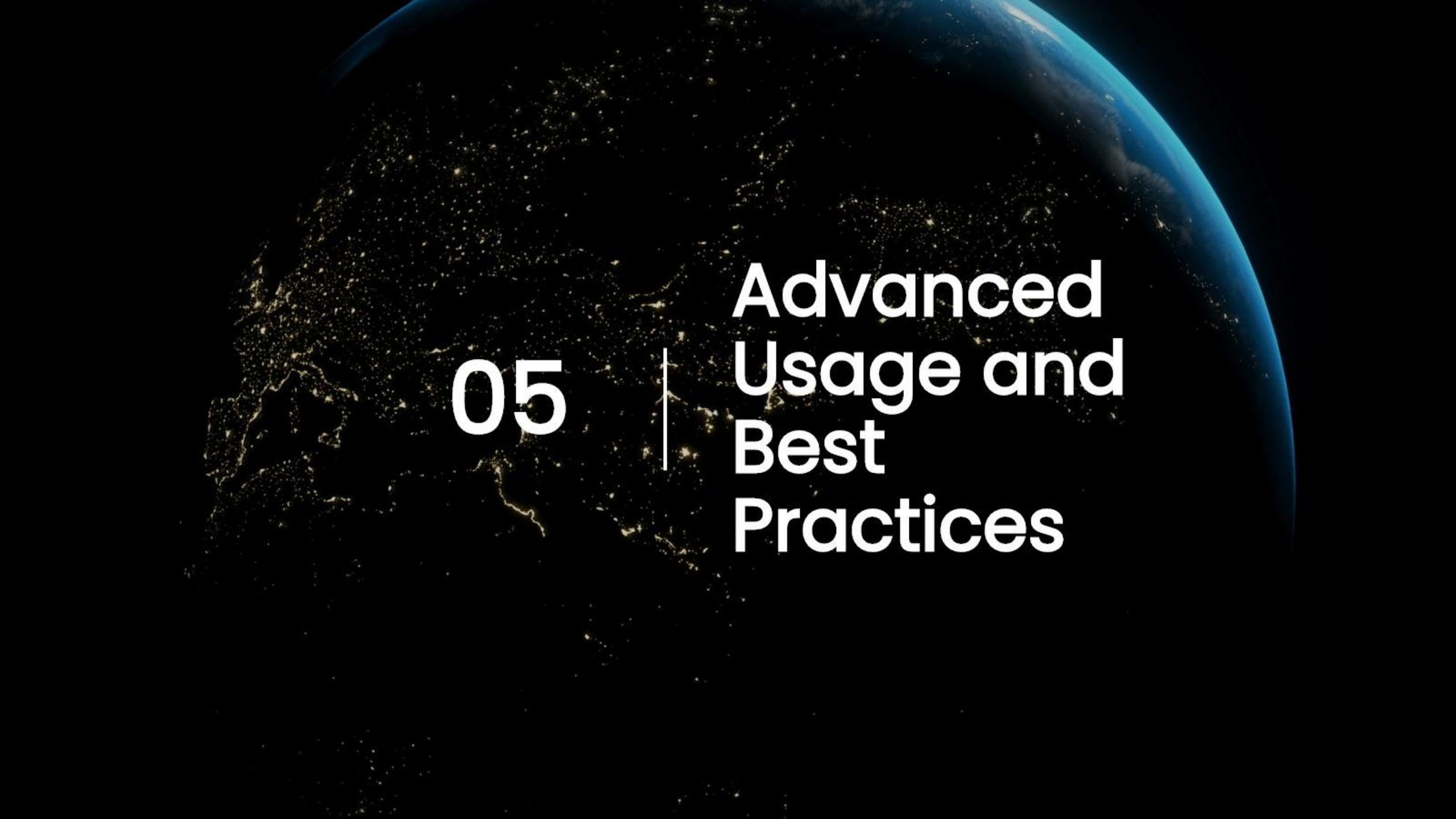
GitHub is a massive learning resource. Explore popular repositories (trending section, GitHub Explore page), read other developers' code, study their project structure, and learn best practices. Follow accounts of developers you admire. Star repositories to bookmark them for later. Study the README files, code organization, and commit messages of well-maintained projects.

02 Community Engagement

Join communities, read discussions, and contribute comments or ideas. This passive learning accelerates your growth.

03 Active Contribution

As you become more confident, move to active contributions by forking, making improvements, and submitting Pull Requests.



05 | Advanced Usage and Best Practices

Contributing to Open Source Projects

Step 1: Find a Project

Use GitHub's search and filtering to find projects matching your interests and skill level. Look for labels like "good-first-issue" or "beginner-friendly."

Step 2: Understand Project

Read the README, contribution guidelines (usually in CONTRIBUTING.md), and check existing issues and PRs to avoid duplicating work.

Step 3: Fork Repository

Click "Fork" to create your copy. Clone it to your computer and create a branch for your contribution.

Step 4: Make Changes

Fix bugs, add features, or improve documentation. Follow the project's coding style and guidelines.

Step 5: Test & Commit

Test your changes thoroughly, write clear commit messages, and push to your fork.

Step 6: Submit Pull Request

Open a PR to the original project, explain your contribution, and respond to feedback. Be patient—maintain often

Project Management with GitHub

GitHub's Built-in Tools

Use GitHub's built-in tools for project management: Create a "Projects" board (accessible from your repo) to organize issues and PRs using Kanban boards (To Do, In Progress, Done). Use milestones to group related issues and PRs for a specific release or version. Assign issues to team members, use labels to categorize work, and add due dates. Link issues to PRs to track related work.

Team Coordination

Hold regular meetings to review progress, discuss blockers, and plan next steps. GitHub integrates with external tools like Slack and Jira for enhanced workflow coordination.

Introduction to GitHub Actions

Common Uses

Running tests automatically (Continuous Integration)
Deploying code to servers (Continuous Deployment)
Checking code quality
Sending notifications

How Actions Work

GitHub Actions automates workflows—tasks that run automatically when certain events occur (e.g., when code is pushed or a PR is opened)
Actions are defined in YAML files stored in a ".github/workflows" folder
Pre-built actions are available in the GitHub Marketplace

Getting Started

For beginners, explore simple actions like auto-testing or code linting
As you advance, automate your entire development pipeline

GitHub Best Practices

01 FIRST

Commit Messages

Write clear, descriptive commit messages (e.g., "Add user authentication feature" instead of "Fix stuff"). Use imperative mood (e.g., "Add" not "Added"). Keep messages concise but informative.

02 SECOND

Documentation

Maintain updated README files, code comments for complex logic, and contribution guidelines. Good documentation reduces support questions and helps new contributors.

03 THIRD

Consistent Workflow

Follow a consistent branching strategy (e.g., main for production, develop for development). Use naming conventions for branches. Review and approve PRs before merging. Maintain code quality standards.

04 FOURTH

Security

Never commit sensitive data (passwords, API keys, tokens). Use environment variables and .gitignore to exclude sensitive files. Regularly review who has access to your repositories.

Building Your Developer Portfolio with GitHub

Consistent Contributions

Your GitHub presence is your portfolio. Commit to consistent contributions—make meaningful commits regularly, even on personal projects.

Documentation & Demos

Include projects with clear documentation and live demos (linked in README).

Professional Profile

Maintain a professional profile: professional photo, clear bio, pinned repositories.

Quality Priority

Quality over quantity—a few well-executed projects impress more than many unfinished ones.

Showcase Diverse Skills

Showcase diverse skills: web development, data science, mobile apps, open source contributions.

Demonstrate Collaboration

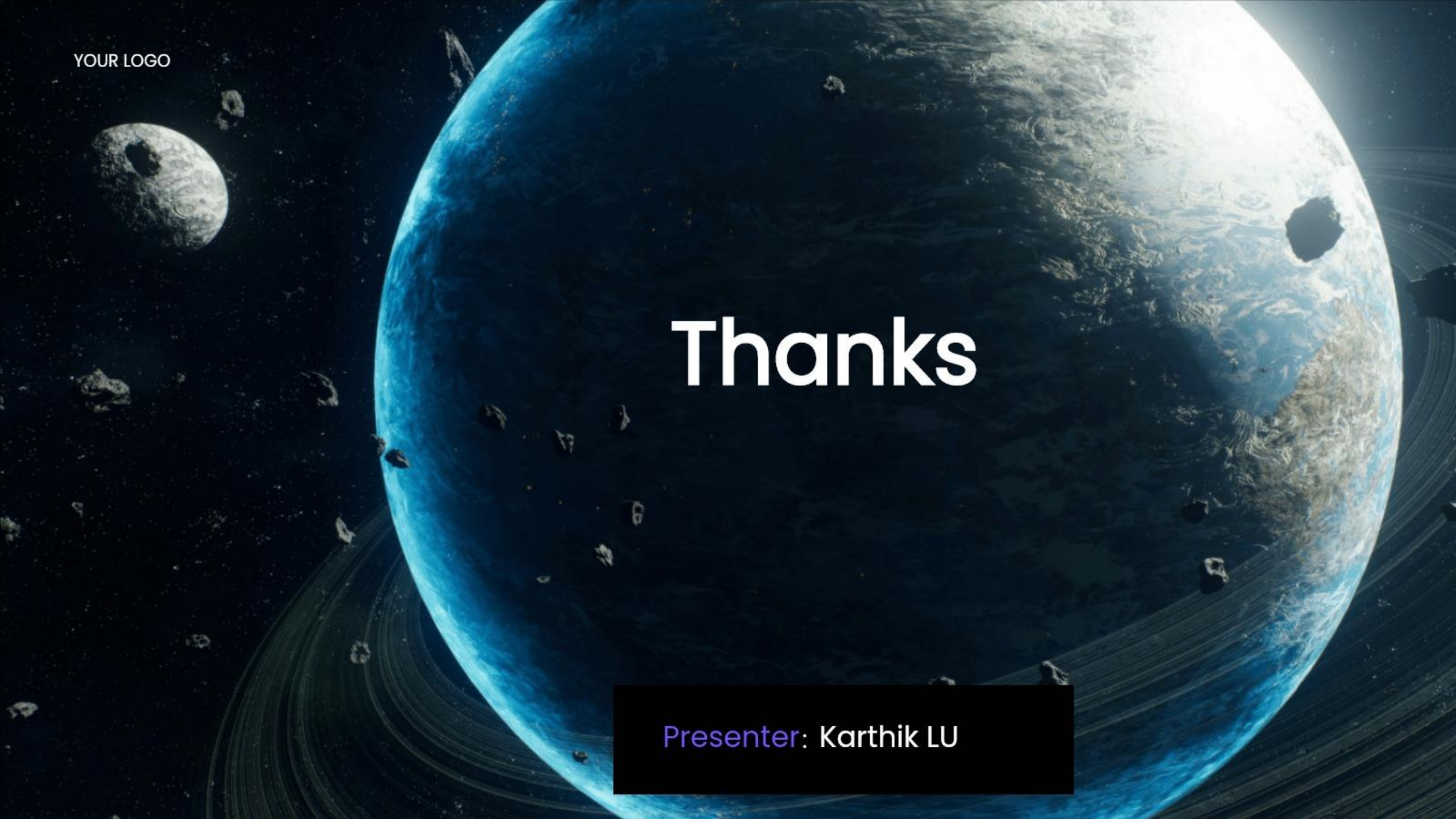
Demonstrate collaboration: show evidence of working with others, reviewing code, and discussing solutions.

Live Projects

Link to live projects (GitHub Pages sites, deployed applications).

Community Engagement

Engage with the community: contribute to open source, answer questions, and share knowledge.


YOUR LOGO
Thanks
Presenter: Karthik LU